DOCUMENTO EDUCATIVO · SPRINT 2

Guía completa del modelo de detección de somnolencia

Cómo se entrenó la red neuronal, qué hacen los filtros, qué significan las métricas y por qué a veces se equivoca. Explicado paso a paso, como si nunca hubieras visto una CNN.

PROYECTO
GuardDrive AI · Detección de Somnolencia

MODELO
CNN_Driver_Drowsiness_Detector

DATASET
Driver Drowsiness Dataset (DDD)

ENTRADA DEL MODELO
Imagen RGB de 64 × 64 píxeles

PARÁMETROS
36,737 pesos entrenables

FECHA DEL DOCUMENTO
23 de octubre de 2026

Deep Learning · Visión por Computadora

Documento generado a partir de train.py, app.py y metrics.json

Índice

Este documento está pensado para que puedas entender el sistema completo sin conocimientos previos. Primero veremos la idea general; después abriremos la “caja negra” capa por capa, filtro por filtro, y finalmente interpretaremos los números.

1. **1.** La idea en una frase
2. **2.** Cómo ve una computadora una foto
3. **3.** Qué es una CNN, explicado con una fábrica de detectives
4. **4.** El dataset DDD: las fotos con las que aprendió
5. **5.** Preparación de las imágenes
6. **6.** La arquitectura, capa por capa
7. **7.** Los filtros: el corazón del modelo
8. **8.** Cómo fue el entrenamiento, época por época
9. **9.** Las métricas: qué significan y cómo se leen
10. **10.** Cómo usa el modelo la aplicación
11. **11.** Limitaciones y seguridad
12. **12.** Qué se podría mejorar
13. **13.** Glosario para principiantes
14. **14.** Resumen para recordar

Una advertencia importante desde el inicio

Este modelo es un proyecto académico y una demostración tecnológica. **No es un dispositivo médico ni un sistema certificado de seguridad vial.** Puede ayudar a entender cómo funciona una CNN, pero no debe usarse como único mecanismo para evitar un accidente.

“Una CNN no “ve” ojos ni caras como nosotros. Ve números, detecta patrones y aprende a asociarlos con una respuesta.”

1. La idea en una frase

El sistema recibe una foto de una persona y devuelve un número entre 0 y 1. Ese número es la **probabilidad estimada de que la persona esté somnolienta**. Si la probabilidad es mayor o igual a 0.5, la aplicación muestra una alerta.



CLASE 0
Non Drowsy
Despierto / alerta

CLASE 1
Drowsy
Somnoliento / fatigado

SALIDA
Sigmoid
Un único número entre 0 y 1

¿Por qué se eligió una CNN y no otra cosa?

Una CNN (red neuronal convolucional) está diseñada para trabajar con imágenes. En lugar de mirar cada píxel por separado, aprende a reconocer patrones locales: bordes, contrastes, curvas, sombras y formas. Esos patrones se van combinando en capas hasta formar una idea más abstracta: "esta imagen se parece más a una persona somnolienta que a una persona despierta".

> **La idea clave**
> La red no fue programada con reglas del tipo "si los ojos están cerrados, entonces somnoliento". Nadie escribió esas reglas. La red **aprendió sus propios filtros** mirando miles de ejemplos etiquetados.

Lo que sí hace y lo que no hace

Sí hace	No hace
Clasifica una imagen completa como "alerta" o "somnoliento" con una probabilidad.	No mide directamente el porcentaje de apertura de los ojos.
Puede usarse con una foto subida o con frames de una cámara web.	No detecta la cara ni recorta los ojos en la versión actual.
En modo cámara, usa una ventana de varios frames para reducir falsas alarmas.	No es un modelo temporal: no "recuerda" el video por sí mismo.

2. Cómo ve una computadora una foto

Para nosotros una foto es una escena. Para la computadora es una tabla gigante de números. Cada número representa la intensidad de un color en un punto diminuto llamado **píxel**.

¿Qué es un píxel?

Imagina una hoja cuadriculada. Cada cuadradito es un píxel. En una foto en color, cada píxel tiene tres valores: uno para el rojo (R), uno para el verde (G) y uno para el azul (B). Cada valor suele ir de 0 a 255:

Canal	Qué representa	Ejemplo
R — Rojo	Cantidad de rojo en ese punto	255 = rojo máximo; 0 = sin rojo
G — Verde	Cantidad de verde en ese punto	255 = verde máximo; 0 = sin verde
B — Azul	Cantidad de azul en ese punto	255 = azul máximo; 0 = sin azul

Si la imagen tiene 64 píxeles de alto y 64 de ancho, y cada píxel tiene 3 canales, la computadora ve una caja de números de:

```
64 × 64 × 3 = 12,288 números
```

Cada número está entre 0 y 255. Antes de entrar a la red, el modelo los convierte a una escala entre 0 y 1. Eso se hace con una capa llamada **Rescaling(1/255)**. ¿Para qué? Para que todos los valores estén en un rango parecido y el entrenamiento sea más estable.

> **Analogía**
> Es como si tuvieras una foto hecha con 12,288 perillas de color. La red no ve la foto completa de golpe: va mirando pequeñas ventanas de perillas y anotando qué patrones aparecen.

¿Por qué 64 × 64 y no la foto original?

Las imágenes originales del dataset DDD son de 227 × 227 píxeles. Usar ese tamaño implica muchos más números y más cómputo. Para un MVP que debe responder rápido en CPU, se redujo a 64 × 64. Es una decisión de compromiso:

Ventaja de 64 × 64

Es mucho más rápido y ocupa menos memoria. Permite entrenar y hacer inferencia en un computador normal.

Desventaja de 64 × 64

Se pierden detalles finos, como el grado exacto de apertura de un párpado o pequeños reflejos. Eso puede afectar la precisión.

Lo que realmente entra a la red

La app toma la imagen, la convierte a RGB, la redimensiona a 64 × 64 y la convierte en un tensor con forma **(1, 64, 64, 3)**. El “1” significa “un ejemplo a la vez”. Luego la capa Rescaling divide todos los valores entre 255.

```
Imagen RGB 227×227 → redimensionar → 64×64×3 → dividir entre 255 → valores en [0.0, 1.0]
```

3. Qué es una CNN, explicado con una fábrica de detectives

Imagina una fábrica con varios detectives. Cada detective mira la misma foto, pero con una lupa distinta. El primero busca líneas y bordes. El segundo combina esas líneas para encontrar formas. El tercero combina las formas para reconocer algo más complejo. Al final, un jefe de equipo suma todas las pistas y decide: “esta persona parece somnolienta”.

Detective 1 · Bordes
Mira contrastes: el borde del párpado, la línea de la boca, un cambio brusco de luz. Sus filtros son pequeños y simples.

Detective 2 · Formas
Combina bordes para reconocer formas: un ojo entreabierto, una comisura caída, una sombra con forma de ojera.

Detective 3 · Sospecha
Combina formas y texturas para construir una idea de “cara cansada”. No necesita decir exactamente qué parte es; le basta con el patrón general.

Las piezas básicas de una CNN

Pieza	Qué hace	Analogía
Convolución	Aplica un filtro pequeño sobre toda la imagen y produce un mapa de activaciones.	Pasar una lupa por la foto y anotar dónde aparece el patrón.
Filtro / kernel	Pequeña matriz de números que se multiplica por los píxeles.	El “sello” o la “lupa” que busca un patrón concreto.
ReLU	Deja los valores positivos y convierte los negativos en cero.	“Si no hay señal, no pasa nada; si hay señal, la dejamos pasar”.
Pooling	Resume regiones de la imagen y reduce su tamaño.	De cada grupo de 4 píxeles, quedarse con el más fuerte.
Dropout	Apaga neuronas al azar durante el entrenamiento.	Estudiar con algunos apuntes tapados para no depender de una sola página.
BatchNorm	Normaliza las activaciones para que no se disparen.	Un profesor que ajusta las notas para que todas estén en la misma escala.
Dense	Combina las características finales para tomar una decisión.	El jefe de equipo que escucha a todos los detectives y decide.
Sigmoid	Convierte la decisión final en un número entre 0 y 1.	Convertir una nota cualquiera en un porcentaje de 0% a 100%.

El flujo completo, en una línea

Imagen 64×64×3 → **Conv 32** 3×3 + ReLU → **BatchNorm** Estabiliza → **Conv 64** 3×3 + ReLU → **MaxPool** 64→32 → **Dropout** 30% → **SepConv 128** 3×3 + 1×1 → **GAP** 128 números → **Dense 64** + Dropout 40% → Probabilidad

Ojo con la palabra “Temp-Conv”

En el código, el primer bloque se llama `block1_temp_conv`. Ese nombre es una etiqueta académica del proyecto, pero la entrada es **una sola imagen**, no una secuencia de video. La parte temporal real está en la app: el buffer de frames consecutivos antes de disparar la alarma.

4. El dataset DDD: las fotos con las que aprendió

El modelo se entrenó con el **Driver Drowsiness Dataset (DDD)**, un conjunto de imágenes de personas en dos situaciones: despiertas y somnolientas. Cada imagen tiene una etiqueta que dice a qué clase pertenece. La red usa esas etiquetas para aprender.

Composición del dataset

Clase	Etiqueta numérica	Imágenes	Porcentaje	Qué muestra
Drowsy (somnoliento)	1	22,348	53.47%	Ojos cerrados o semicerrados, cabeza caída, bostezos, relajación facial.
Non Drowsy (despierto)	0	19,445	46.53%	Ojos abiertos, mirada atenta, expresión neutra o alerta.
Total		**41,793**	**100%**	Imágenes RGB de 227 × 227 píxeles.

> **El dataset está ligeramente desbalanceado**
> Hay un poco más de imágenes de personas somnolientas (53.47%) que despiertas (46.53%). No es un desbalance extremo, pero es una de las razones por las que el modelo tiende a detectar muy bien la somnolencia y, a cambio, genera más falsas alarmas cuando la persona está despierta.

¿Cómo se dividieron las imágenes?

El script usa una división clásica en tres partes. La idea es que el modelo nunca vea el examen final durante el entrenamiento.

Conjunto	Porcentaje	Para qué sirve	Analogía
Entrenamiento (train)	70%	La red ve las imágenes y ajusta sus pesos.	Estudiar con el libro y los ejercicios.
Validación (val)	15%	Se usa para vigilar si está aprendiendo o memorizando; guía los callbacks.	Un simulacro de examen para corregir el estudio.
Prueba (test)	15%	Se usa una sola vez al final para medir el rendimiento real.	El examen final con preguntas que nunca vio.

> **Importante: no se usó el dataset completo**
> Para que el entrenamiento fuera rápido en el equipo local, la corrida registrada usó un subconjunto configurado con `max_samples = 9600`. El conjunto de prueba efectivo terminó con **1,152 imágenes**. Esto significa que los resultados son válidos para esa corrida, pero **no representan el rendimiento máximo posible** si se entrenara con las 41,793 imágenes.

Distribución del conjunto de prueba

Clase	Imágenes de prueba	Porcentaje del test
Non Drowsy	542	47.05%
Drowsy	610	52.95%
Total	**1,152**	**100%**

El conjunto de prueba quedó razonablemente balanceado: 542 imágenes de personas despiertas y 610 de personas somnolientas. Eso permite calcular métricas por clase sin que una clase aplaste a la otra.

5. Preparación de las imágenes

Antes de entrenar, las imágenes pasan por un proceso sencillo pero importante. La red no recibe la foto tal como está: recibe un tensor numérico del tamaño y la escala correctos.

Los pasos, uno por uno

#	Paso	Qué ocurre	Por qué importa
1	**Leer la imagen**	Se carga el archivo JPG/PNG o el frame de la cámara.	Es la materia prima.
2	**Convertir a RGB**	Se eliminan canales extra (por ejemplo, transparencia) y se asegura el orden rojo-verde-azul.	Todas las imágenes deben tener el mismo formato.
3	**Redimensionar**	Se lleva a 64 × 64 píxeles con interpolación.	La red tiene una entrada fija.
4	**Crear el tensor**	Se agrega una dimensión para el lote: (1, 64, 64, 3).	Keras espera un lote de ejemplos.
5	**Normalizar**	La capa Rescaling divide entre 255 y deja los valores entre 0 y 1.	Estabiliza el entrenamiento.

```
pixel original: 0 … 255
pixel normalizado: pixel / 255 → 0.0 … 1.0
```

Lo que no se hizo en esta versión

No se recortó la cara

La app redimensiona la imagen completa. No usa el archivo `haarcascade_frontalface_default.xml` para detectar y recortar el rostro. Ese archivo está en el repositorio, pero la versión actual no lo utiliza.

No se aumentaron los datos

No se aplicaron rotaciones, espejos, cambios de brillo ni recortes aleatorios durante el entrenamiento. Eso habría ayudado a generalizar mejor.

La consecuencia práctica

Si la cara ocupa una parte pequeña de la foto, al reducir todo a 64 × 64 los ojos pueden quedar con muy pocos píxeles. El modelo funciona mejor cuando el rostro está relativamente grande y centrado. Es una limitación importante para uso real.

6. La arquitectura, capa por capa

La red tiene 11 capas y 36,737 parámetros entrenables. Es pequeña a propósito: debe cargar rápido, inferir rápido y poder ejecutarse en un computador normal. A continuación se explica cada capa, qué forma tiene su salida y para qué está.

Capa	Tipo	Salida	Params	Propósito
Entrada	InputLayer	(None, 64, 64, 3)	0	Recibe la imagen RGB cruda.
Normalización	Rescaling(1/255)	(None, 64, 64, 3)	0	Lleva los píxeles de 0–255 a 0–1.
Bloque 1	Conv2D 32, 3×3, ReLU	(None, 64, 64, 32)	896	Detecta bordes, contrastes y micro-transiciones.
BatchNorm 1	BatchNormalization	(None, 64, 64, 32)	128	Estabiliza media y varianza de las activaciones.
Bloque 2	Conv2D 64, 3×3, ReLU	(None, 64, 64, 64)	18,496	Combina bordes en formas faciales: ojos, boca, sombras.
Pooling 2	MaxPooling2D 2×2	(None, 32, 32, 64)	0	Reduce de 64×64 a 32×32; conserva lo más fuerte.
Dropout 1	Dropout 0.30	(None, 32, 32, 64)	0	Apaga 30% de conexiones para evitar sobreajuste.
Bloque 3	SeparableConv2D 128, 3×3	(None, 32, 32, 128)	8,896	Combina rasgos de alto nivel con bajo costo computacional.
Pooling 3	GlobalAveragePooling2D	(None, 128)	0	Convierte cada mapa de 32×32 en un solo número.
MLP intermedio	Dense 64, ReLU	(None, 64)	8,256	Combina las 128 características en 64 conceptos útiles.
Dropout 2	Dropout 0.40	(None, 64)	0	Regularización antes de la decisión final.
Salida	Dense 1, Sigmoid	(None, 1)	65	Entrega P(somnoliento | imagen) entre 0 y 1.

Total: 36,737 parámetros

En float32, los pesos ocupan aproximadamente **143.5 KiB**. El archivo `model_ddd.keras` pesa unos **484 KiB** porque además guarda metadatos, configuración y estado adicional. Es un modelo muy liviano.

Los tres bloques convolucionales

Bloque 1 · 32 filtros

Qué busca: bordes, líneas, cambios de brillo, contrastes pequeños.

Por qué 32: es suficiente para capturar patrones simples sin disparar el costo.

Bloque 2 · 64 filtros

Qué busca: combinaciones de bordes: formas de ojos, párpados, boca, sombras.

Por qué 64: al duplicar los filtros, la red puede especializarse en más patrones.

Bloque 3 · 128 filtros

Qué busca: combinaciones de alto nivel asociadas al cansancio facial.

Por qué 128: más capacidad para el concepto final, pero con convolución separable para que sea barato.

¿Por qué la red se hace más “profunda” pero más pequeña?

Al inicio la imagen es grande (64 × 64) pero tiene pocos canales (3). Después de la primera convolución hay 32 canales, luego 64 y luego 128. A la vez, el MaxPooling reduce el tamaño espacial de 64 × 64 a 32 × 32. La red cambia “resolución espacial” por “riqueza de patrones”: mira menos lugares, pero cada lugar está descrito con más detalle abstracto.

```
64×64×3 → 64×64×32 → 64×64×64 → 32×32×64 → 32×32×128 → 128 → 64 → 1
```

7. Los filtros: el corazón del modelo

Si tuvieras que entender una sola cosa de este documento, sería esta: **un filtro es una pequeña matriz de números que se desliza por la imagen para detectar un patrón**. La red tiene muchos filtros y aprende sus números durante el entrenamiento.

¿Qué es exactamente un filtro?

Un filtro (también llamado *kernel*) suele ser una matriz de 3 × 3. Cada casilla contiene un número que la red puede ajustar. Al pasar el filtro por la imagen, se multiplican los números del filtro por los píxeles correspondientes y se suman. El resultado indica "cuánto se parece esta zona al patrón que busca el filtro".

Ejemplo de parche de imagen

	Col 1	Col 2	Col 3
Fila 1	10	10	90
Fila 2	10	10	90
Fila 3	10	10	90

Imagina una zona con una transición vertical: izquierda oscura, derecha clara.

Filtro que detecta bordes verticales

	Col 1	Col 2	Col 3
Fila 1	-1	0	1
Fila 2	-1	0	1
Fila 3	-1	0	1

Este filtro resta la columna izquierda y suma la derecha.

La cuenta, paso a paso

```
Fila 1: (10×-1) + (10×0) + (90×1) = 80
Fila 2: (10×-1) + (10×0) + (90×1) = 80
Fila 3: (10×-1) + (10×0) + (90×1) = 80
Resultado = 80 + 80 + 80 = 240
```

Un resultado alto (240) significa "aquí hay un borde vertical fuerte". Si la zona fuera uniforme, por ejemplo todo 10, el resultado sería 0: no hay borde. Así, el filtro convierte una zona de la imagen en un número que mide la presencia de un patrón.

> **Lo más importante**
> Nosotros escribimos el filtro de ejemplo para explicarlo, pero **la red no usa filtros escritos a mano**. Al comenzar el entrenamiento, sus filtros son números aleatorios. Con cada error, la retropropagación ajusta esos números hasta que empiezan a detectar patrones útiles.

¿Cómo aprende los filtros?

1. **Empieza al azar:** los 32 filtros del primer bloque son matrices con números aleatorios.
2. **Mira un lote de imágenes:** por ejemplo, 64 imágenes etiquetadas.

3. **Hace una predicción:** calcula una probabilidad para cada imagen.
4. **Mide el error:** compara la predicción con la etiqueta real usando la pérdida.
5. **Retropropaga:** calcula cómo cambiar cada número para reducir el error.
6. **Actualiza:** el optimizador Adam ajusta los filtros y los pesos de las capas densas.
7. **Repite:** miles de veces, época tras época.

Con el tiempo, los filtros del primer bloque tienden a especializarse en bordes y contrastes; los del segundo, en formas; los del tercero, en combinaciones más complejas. No podemos garantizar que un filtro concreto sea “el filtro del ojo”, pero el conjunto aprende representaciones útiles para la tarea.

Los filtros de este modelo, en números

Bloque	Cantidad	Tamaño del filtro	Canales de entrada	Parámetros	Qué aprende en la práctica
Conv2D 1	32	3 × 3	3 (RGB)	896	Bordes, líneas, cambios de luz, contrastes locales.
Conv2D 2	64	3 × 3	32	18,496	Formas: contornos de ojos, párpados, boca, sombras.
SeparableConv2D	128	3 × 3 + 1 × 1	64	8,896	Combinaciones de alto nivel; patrones de cansancio.

¿Por qué 3 × 3?

Un filtro de 3 × 3 es pequeño y barato, pero al apilar varias capas la red puede ver regiones cada vez más grandes. Es como mirar una foto con una lupa pequeña, pero pasar la lupa muchas veces y combinar lo visto. Además, dos capas de 3 × 3 cubren una zona de 5 × 5 con menos parámetros que un solo filtro de 5 × 5.

¿Qué significan “padding same” y “stride 1”?

Padding “same”

Antes de aplicar el filtro, se rellena la imagen con ceros en los bordes para que la salida tenga el mismo tamaño que la entrada. Así no se pierden los bordes de la foto.

Stride 1

El filtro se mueve un píxel a la vez. Es una exploración densa: mira todos los lugares posibles. Un stride mayor saltaría píxeles y reduciría el tamaño de salida.

¿Qué pasa después de cada filtro?

Operación	Qué hace	Por qué está ahí
ReLU	Convierte los valores negativos en 0 y deja los positivos iguales.	Introduce no linealidad; la red puede aprender relaciones más complejas.
BatchNorm	Normaliza las activaciones de cada lote.	Evita que los valores se disparen y acelera la convergencia.
MaxPooling 2×2	De cada bloque de 2×2 se queda con el valor máximo.	Reduce el tamaño y conserva la señal más fuerte; da algo de robustez a pequeños desplazamientos.
Dropout 0.30	Apaga al azar el 30% de las conexiones durante el entrenamiento.	Evita que la red dependa de un solo patrón y memorice.
GlobalAveragePooling	Promedia cada mapa de 32×32 y lo convierte en un número.	Reduce drásticamente los parámetros antes del clasificador.
Dense 64 + Dropout 0.40	Combina las 128 características en 64 y regulariza.	Permite razonar sobre las características, no solo sobre píxeles.
Dense 1 + Sigmoid	Produce un único valor entre 0 y 1.	Es la probabilidad de somnolencia que usa la app.

La convolución separable, explicada fácil

Una convolución normal aplica un filtro que mezcla a la vez el espacio (3 × 3) y los canales (por ejemplo, 64 canales). La convolución separable lo divide en dos pasos:

1. **Depthwise:** aplica un filtro espacial a cada canal por separado. Es como revisar cada “capa” de la imagen por su cuenta.
2. **Pointwise:** usa filtros de 1 × 1 para mezclar la información de los canales. Es como combinar los informes de cada detective.

Esto reduce mucho el número de operaciones. En el proyecto se usa para que el modelo sea liviano y pueda correr en tiempo real.

> **Los filtros no son “ojos mágicos”**
> La red no tiene un filtro llamado “ojo cerrado”. Lo que aprende es un conjunto de patrones estadísticos que, combinados, se correlacionan con la etiqueta “Drowsy”. Por eso puede fallar con sombras, gafas, manos, ángulos raros o personas que no se parecen a las del dataset.

8. Cómo fue el entrenamiento, época por época

Entrenar es un ciclo de prueba y error. La red mira ejemplos, hace predicciones, se equivoca, calcula cuánto se equivocó y ajusta sus números. Ese proceso se repite muchas veces.

Las piezas del entrenamiento

Concepto	Qué significa	Valor usado
Función de pérdida	Un número que mide cuánto se equivocó la red. Cuanto más alto, peor.	Binary Crossentropy
Optimizador	El algoritmo que decide cómo cambiar los pesos para reducir la pérdida.	Adam
Tasa de aprendizaje	Qué tan grande es el paso que da al ajustar los pesos.	0.001
Batch	Grupo de imágenes que se procesa antes de actualizar los pesos.	64 imágenes
Época	Una pasada completa por el conjunto de entrenamiento.	6 épocas en la corrida registrada
Semilla	Número para que el azar sea reproducible.	42
Tiempo	Duración total del entrenamiento.	168.48 segundos ≈ 2 min 48 s

La pérdida, explicada como un castigo

La red produce una probabilidad **p** de que la imagen sea somnolienta. La etiqueta real **y** es 1 si es Drowsy y 0 si es Non Drowsy. La pérdida de entropía cruzada binaria se calcula así:

```
L = −[ y · log(p) + (1 − y) · log(1 − p) ]
```

Si la respuesta correcta es 1 y la red dice 0.99, la pérdida es pequeña. Si dice 0.01, la pérdida es enorme. Si la respuesta correcta es 0 y la red dice 0.99, también es enorme. El entrenamiento busca que ese castigo sea cada vez menor.

¿Qué es la retropropagación?

Después de calcular el error, la red mira hacia atrás, capa por capa, y calcula cuánto contribuyó cada peso al error. Luego los ajusta un poquito en la dirección que reduce el error. Es como un equipo de fútbol que revisa el video del partido y corrige los movimientos que salieron mal.

> *“La red no aprende porque se le explique la regla. Aprende porque ajusta millones de números después de equivocarse.”*

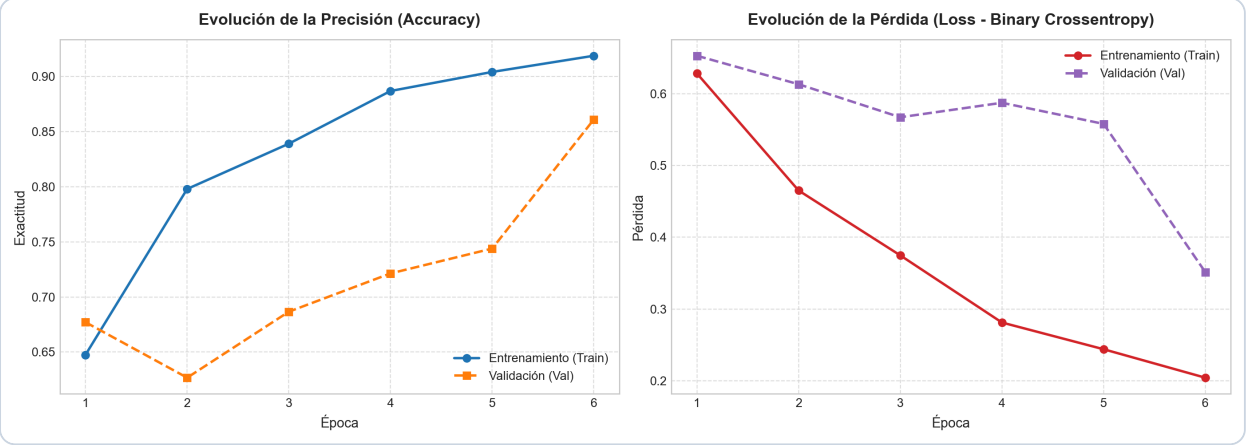
Los callbacks: los “entrenadores asistentes”

Callback	Qué vigila	Qué hace
EarlyStopping	val_loss	Si la validación deja de mejorar durante 4 épocas, detiene el entrenamiento y recupera los mejores pesos.
ReduceLROnPlateau	val_loss	Si no mejora durante 2 épocas, reduce la tasa de aprendizaje a la mitad (factor 0.5), con mínimo 0.00001.
ModelCheckpoint	val_accuracy	Guarda el modelo cada vez que mejora la exactitud de validación.

> **¿Se activaron los callbacks?**
> La corrida registrada completó 6 épocas. El script permite hasta 8 por defecto, pero los callbacks pueden detenerlo antes. En este caso, la historia muestra 6 épocas y una mejora fuerte en la última, así que no podemos afirmar que EarlyStopping lo haya detenido; simplemente sabemos que se completaron 6. Lo importante es que el modelo final quedó guardado en `model_ddd.keras`.

La historia real del entrenamiento

Época	Train loss	Val loss	Train accuracy	Val accuracy
1	0.6284	0.6530	64.75%	67.71%
2	0.4652	0.6133	79.80%	62.67%
3	0.3747	0.5672	83.91%	68.66%
4	0.2811	0.5874	88.69%	72.14%
5	0.2439	0.5580	90.42%	74.39%
6	**0.2042**	**0.3515**	**91.89%**	**86.11%**



Curvas reales del proyecto: exactitud (izquierda) y pérdida (derecha), comparando entrenamiento y validación.

¿Cómo se leen estas curvas?

- **La exactitud de entrenamiento sube de 64.75% a 91.89%.** La red está aprendiendo los ejemplos que ve.

- **La exactitud de validación sube de 67.71% a 86.11%.** También mejora con datos que no usó para ajustar los pesos, aunque con altibajos.
- **La pérdida de entrenamiento baja de 0.6284 a 0.2042.** Se equivoca cada vez menos en el entrenamiento.
- **La pérdida de validación baja de 0.6530 a 0.3515.** Mejora, pero queda por encima de la de entrenamiento.

> **Hay una brecha entre train y validación**
>
> Al final, train accuracy (91.89%) es mayor que val accuracy (86.11%), y train loss (0.2042) es menor que val loss (0.3515). Esa brecha sugiere cierto **sobreajuste**: la red se desempeña mejor en lo que vio que en lo nuevo. Con más datos, aumento de datos (augmentation) o regularización adicional, la brecha podría reducirse.

La orden que se usó para reproducir la corrida

```
python train.py --epochs 6 --batch_size 64 --max_samples 9600
```

Los resultados pueden variar ligeramente según el equipo, la versión de TensorFlow y el azar, aunque se fijó la semilla 42. Para entrenar con todo el dataset se usaría `--max_samples 0` y más épocas, pero tardaría mucho más.

9. Las métricas: qué significan y cómo se leen

Las métricas son la forma de medir si el modelo es bueno. No basta con mirar una sola: cada una cuenta una parte de la historia. Aquí usaremos el conjunto de prueba de 1,152 imágenes que el modelo nunca vio durante el entrenamiento.

ACCURACY
86.46%
De cada 100 imágenes, acierta 86.46.

PRECISION (DROWSY)
81.35%
Cuando dice "somnoliento", acierta 81.35 de cada 100 veces.

RECALL (DROWSY)
96.56%
De cada 100 somnolientos reales, detecta 96.56.

F1-SCORE (DROWSY)
88.31%
Equilibrio entre precisión y recall.

AUC-ROC
94.86%
Capacidad de separar somnolientos de despiertos.

TEST
1,152
542 despiertos + 610 somnolientos.

Primero, la tabla de confusión

La matriz de confusión es la base de casi todas las métricas. Cruza lo que **era** la imagen con lo que el modelo **predijo**.

	El modelo predijo: Non Drowsy	El modelo predijo: Drowsy
La realidad era: Non Drowsy	**407** Verdadero Negativo (TN) 75.09%	**135** Falso Positivo (FP) 24.91%
La realidad era: Drowsy	**21** Falso Negativo (FN) 3.44%	**589** Verdadero Positivo (TP) 96.56%

Lo que hace bien

✓ Detecta 589 de 610 personas somnolientas: **96.56% de recall**.

✓ Reconoce 407 de 542 personas despiertas: **75.09% de especificidad**.

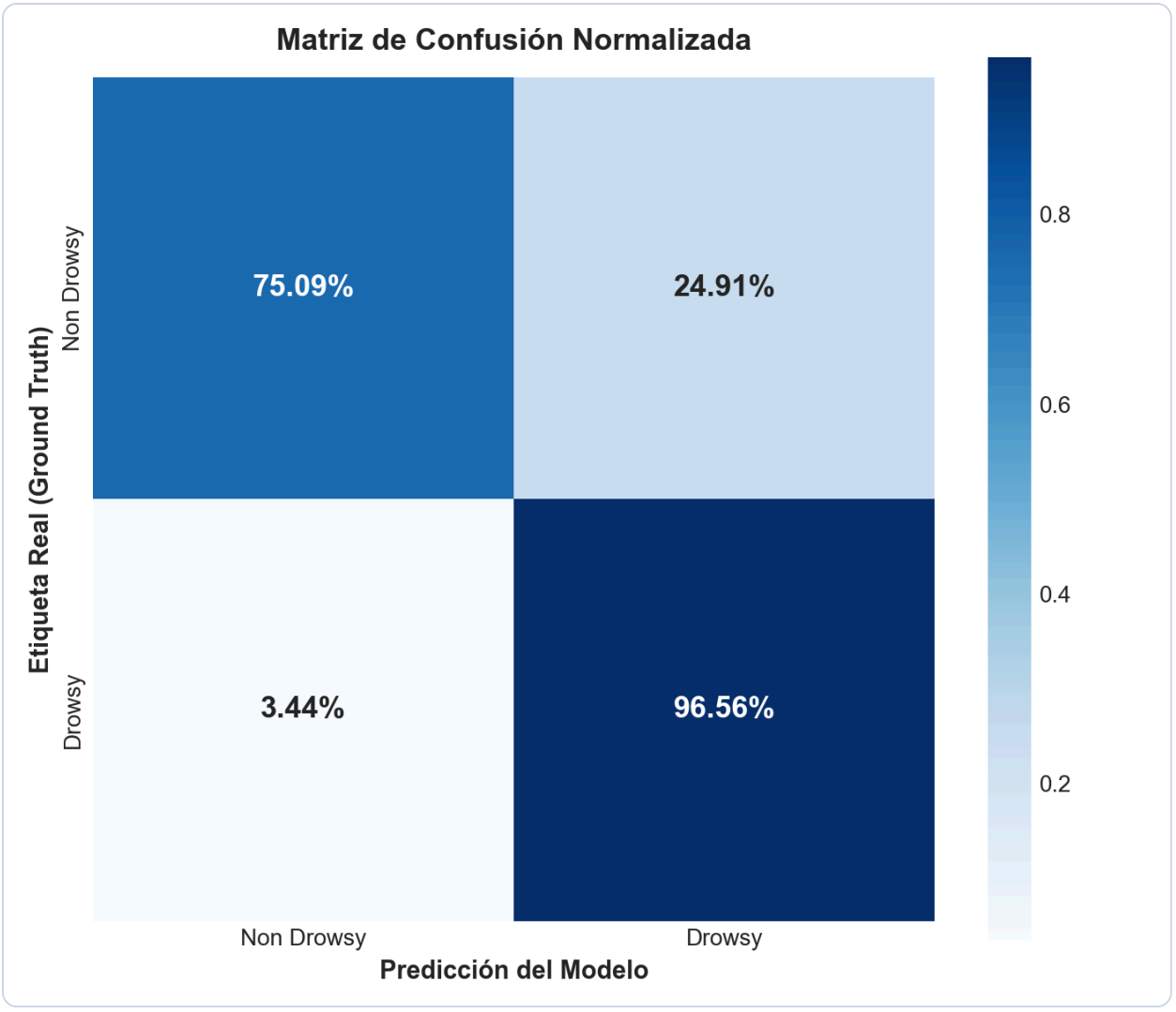
✓ En total, acierta 996 de 1,152 imágenes: **86.46%**.

Lo que falla

✗ Se le escapan 21 personas somnolientas: **3.44% de falsos negativos**.

✗ Marca como somnolientas a 135 personas despiertas: **24.91% de falsos positivos**.

! La mayoría de los errores son falsas alarmas, no somnolencia no detectada.



Matriz de confusión normalizada del proyecto. Cada fila suma 100%: muestra qué porcentaje de cada clase real fue clasificado correctamente.

Accuracy: el porcentaje de aciertos

Accuracy = (aciertos totales) / (total de imágenes). Aquí: (407 + 589) / 1,152 = 0.8646 = 86.46%. Es la métrica más fácil de entender, pero puede engañar si las clases están muy desbalanceadas. En este caso el test está bastante balanceado, así que es una referencia útil.

> **Analogía**
> Si tuvieras 100 preguntas de un examen y contestaras bien 86, tu “accuracy” sería 86%. Pero eso no te dice si fallaste las preguntas más importantes.

Precision: cuando dice “hay somnolencia”, ¿cuánto acierta?

Precision (Drowsy) = TP / (TP + FP) = 589 / (589 + 135) = 0.8135 = 81.35%.

De todas las veces que el modelo dijo “Drowsy”, el 81.35% eran realmente somnolientas. El otro 18.65% eran falsas alarmas. Es como una alarma de humo: cuando suena, ¿siempre hay fuego? La precisión responde esa pregunta.

Recall: de todos los somnolientos reales, ¿cuántos detecta?

Recall (Drowsy) = TP / (TP + FN) = 589 / (589 + 21) = 0.9656 = 96.56%.

De las 610 imágenes realmente somnolientas, detectó 589 y se le escaparon 21. Es como una alarma de humo: de todos los incendios reales, ¿cuántos detecta? El recall responde esa pregunta.

> **La prioridad del proyecto**
> Para seguridad, suele ser más importante un recall alto: es peor no detectar a alguien somnoliento que dar una falsa alarma. Este modelo tiene recall 96.56%, así que prioriza no dejar pasar la somnolencia. El costo es que genera muchas falsas alarmas (24.91% de las personas despiertas).

F1-Score: el equilibrio

F1 = 2 × (precision × recall) / (precision + recall). Si precision y recall fueran perfectos (1.0), el F1 sería 1.0. Aquí es 0.8831 = 88.31%. Es una media armónica: castiga mucho cuando una de las dos métricas es baja.

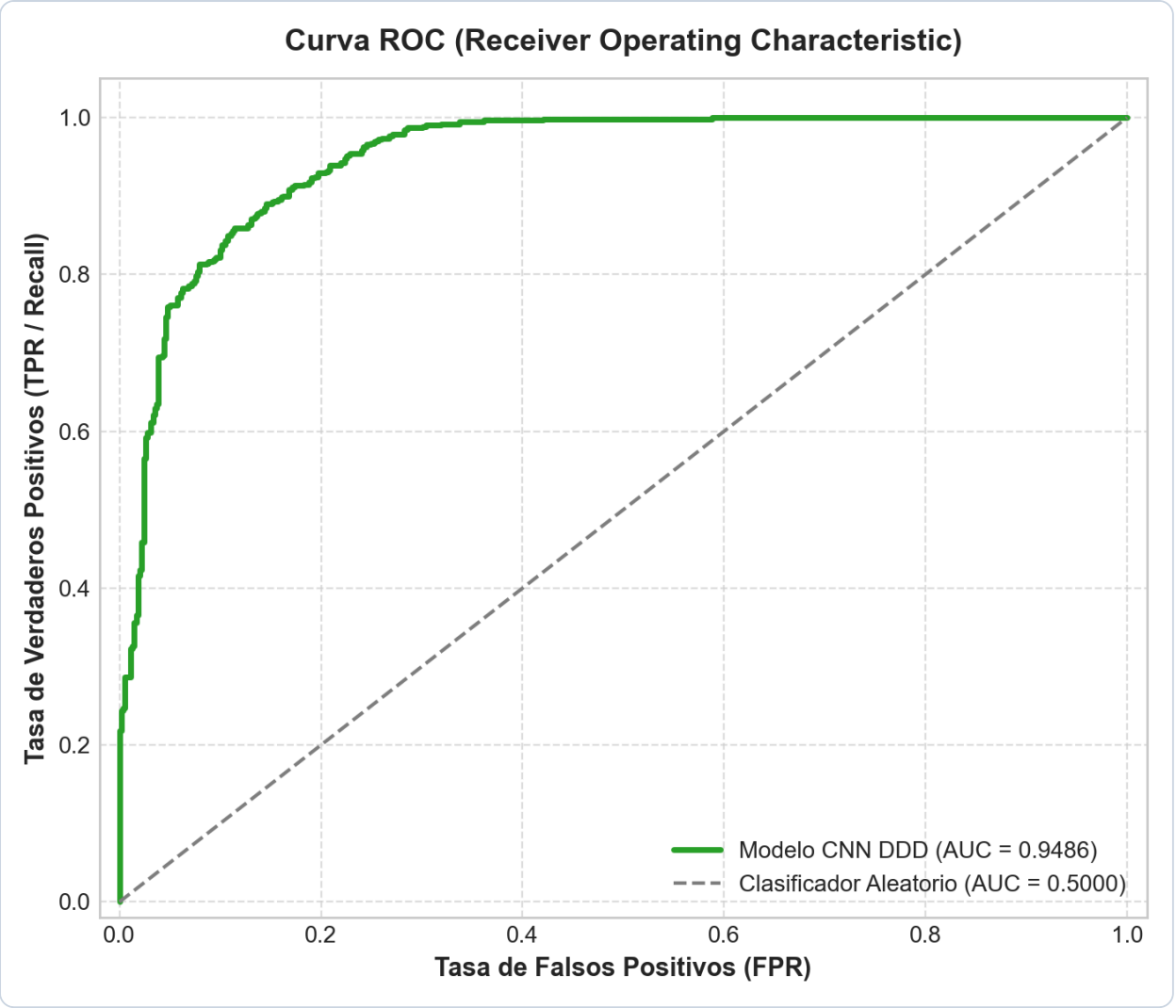
Especificidad: ¿cuántos despiertos reconoce como despiertos?

Especificidad = TN / (TN + FP) = 407 / 542 = 0.7509 = 75.09%. De cada 100 personas despiertas, 75 son reconocidas como despiertas y 25 reciben una falsa alarma. Esta métrica no aparece en las tarjetas principales, pero es clave para entender el rendimiento.

AUC-ROC: la capacidad de ordenar

El AUC mide qué tan bien separa el modelo las dos clases usando la probabilidad, sin depender de un umbral fijo. Un AUC de 0.5 equivale a lanzar una moneda. Un AUC de 1.0 es separación perfecta. El modelo obtuvo **0.9486**.

> *“Si tomamos al azar una persona somnolienta y una despierta, el modelo le asigna una probabilidad de somnolencia más alta a la somnolienta en el 94.86% de los casos.”*



Curva ROC del proyecto. Cuanto más cerca está la curva de la esquina superior izquierda, mejor es el clasificador. La línea gris representa un clasificador aleatorio (AUC 0.5).

Reporte por clase

Clase	Precision	Recall	F1-score	Soporte
Non Drowsy	0.9509 (95.09%)	0.7509 (75.09%)	0.8392 (83.92%)	542
Drowsy	0.8135 (81.35%)	0.9656 (96.56%)	0.8831 (88.31%)	610
Macro avg	0.8822	0.8582	0.8611	1,152
Weighted avg	0.8782	0.8646	0.8624	1,152

¿Por qué la precisión de “Non Drowsy” es alta pero su recall es bajo?

Porque cuando el modelo dice “despierto”, casi siempre acierta (precisión 95.09%), pero se le escapan muchos despiertos que clasifica como somnolientos (recall 75.09%). Esa aparente contradicción es normal en matrices de confusión: las métricas miran desde ángulos distintos.

Resumen honesto del rendimiento

- **Muy bueno detectando somnolencia:** recall 96.56%, solo 21 falsos negativos de 610.

- **Regular confirmando que alguien está despierto:** 24.91% de falsas alarmas.
- **Buen equilibrio general:** F1 88.31%, AUC 94.86%, accuracy 86.46%.
- **No es perfecto:** 156 errores en 1,152 imágenes; la mayoría son falsas alarmas.

10. Cómo usa el modelo la aplicación

El modelo es solo una parte. La app se encarga de recibir la imagen, prepararla, pedirle una predicción y mostrar el resultado de forma comprensible.

Modo “Subir Imagen / Foto de Prueba”

1. El usuario sube una imagen JPG, JPEG o PNG.
2. La app la abre con PIL y la convierte a RGB.
3. La redimensiona a 64 × 64 con OpenCV.
4. Crea un tensor (1, 64, 64, 3) y lo pasa al modelo.
5. El modelo devuelve una probabilidad entre 0 y 1.
6. Si la probabilidad es ≥ 0.50, muestra “Somnolencia detectada”; si no, “Alerta / Despierto”.

```
p = modelo(imagen)
si p ≥ 0.50 → Drowsy
si p < 0.50 → Non Drowsy
```

En este modo no hay buffer temporal: se analiza una sola imagen. Por eso una foto con los ojos cerrados puede dar una probabilidad alta, aunque la persona solo esté parpadeando.

Modo “Cámara Web en Vivo”

1. OpenCV captura frames de la cámara.
2. Cada frame se convierte a RGB y se redimensiona a 64 × 64.
3. El modelo calcula la probabilidad de somnolencia del frame.
4. Si la probabilidad supera el umbral, aumenta un contador de frames consecutivos.
5. Si no, el contador baja.
6. Cuando el contador llega a 10 frames (valor por defecto), se dispara la alarma.



El buffer temporal es una regla externa al modelo. Sirve para evitar que un parpadeo normal dispare la alarma. No es una red recurrente ni un modelo de video: es un contador de frames consecutivos.

Parámetros configurables

Parámetro	Valor por defecto	Qué controla
Umbral de decisión	0.50	Probabilidad mínima para considerar somnolencia.
Ventana temporal	10 frames	Frames consecutivos necesarios para disparar la alarma.
Alarma sonora	Activada	Emite un tono en Windows; en la nube no suele funcionar.
Índice de cámara	0	Qué cámara usa OpenCV.

> **El modo cámara no funciona en Streamlit Cloud**
>
> Streamlit Cloud ejecuta el código en un servidor remoto, no en tu computador. La cámara del servidor no es tu cámara. Por eso, en la nube se debe usar “Subir Imagen / Foto de Prueba”. El modo cámara está pensado para ejecución local.

Lo que la app no hace

- No recorta la cara con Haar Cascade en la versión actual.
- No mide el tiempo real de ojos cerrados (EAR) ni la apertura de la boca (MAR).
- No usa un modelo temporal de video; el buffer es una regla de post-procesamiento.
- No garantiza que la alarma suene en el celular.

11. Limitaciones y seguridad

Entender las limitaciones es tan importante como entender los aciertos. Un modelo con buenas métricas en un dataset puede fallar en el mundo real si las condiciones cambian.

Limitaciones del entrenamiento

Limitación	Qué significa	Consecuencia
Subconjunto de datos	Se usaron hasta 9,600 muestras de las 41,793 disponibles.	El modelo no aprovechó toda la variedad del dataset; los resultados podrían mejorar con más datos.
División aleatoria	Las imágenes se repartieron al azar, no por sujeto.	Si la misma persona aparece en train y test, las métricas pueden ser optimistas.
Sin aumento de datos	No se aplicaron rotaciones, espejos, brillos o recortes.	Puede generalizar peor ante cámaras, ángulos e iluminaciones nuevas.
Desbalance moderado	53.47% Drowsy vs 46.53% Non Drowsy; no se usaron pesos de clase.	El modelo favorece la clase Drowsy: alto recall, pero más falsas alarmas.
Resolución 64×64	Se pierden detalles finos de ojos y párpados.	Casos límite (ojos semicerrados, gafas, sombras) son más difíciles.

Limitaciones del modelo y de la app

Limitación	Qué significa	Consecuencia
Una sola imagen	El modelo no entiende movimiento ni tiempo.	Un parpadeo puede parecer somnolencia; el buffer ayuda, pero no es un modelo temporal.
Sin detección de cara	No recorta el rostro; usa la imagen completa.	Si la cara es pequeña o está descentrada, el rendimiento baja.
Falsas alarmas	24.91% de las personas despiertas fueron marcadas como somnolientas.	Puede ser molesto; se mitiga con el buffer de 10 frames, no se elimina.
Falsos negativos	3.44% de las personas somnolientas no fueron detectadas.	Ningún sistema es perfecto; no debe ser la única protección.
Sesgos del dataset	Las personas, cámaras, luces y escenarios del DDD no representan todo el mundo.	Puede fallar más con personas, edades, tonos de piel, gafas o vehículos distintos.

> **Advertencia de seguridad**
>
> Este sistema es un proyecto académico. **No está certificado para uso vehicular, médico ni de seguridad industrial.** No reemplaza la atención del conductor, las pausas, el descanso ni los sistemas

de seguridad del vehículo. Si alguien tiene sueño, debe detenerse y descansar; ningún modelo justifica seguir conduciendo.

La pregunta que siempre hay que hacer

“¿En qué situaciones este modelo podría equivocarse de una forma peligrosa?”

Por ejemplo: una persona con los ojos cerrados por una foto mal tomada, alguien con gafas oscuras, una cara parcialmente tapada, una iluminación muy baja o una persona que no se parece a las del dataset. Un sistema real necesitaría pruebas controladas, validación con muchos sujetos y un diseño a prueba de fallos.

12. Qué se podría mejorar

El modelo actual es un MVP funcional. Estas son las mejoras con mayor impacto potencial, ordenadas de más simple a más ambiciosa.

1. Usar todo el dataset

Entrenar con las 41,793 imágenes en lugar de 9,600. Más variedad debería mejorar la generalización, aunque requerirá más tiempo y posiblemente una GPU.

2. División por sujeto

Asegurar que la misma persona no aparezca en train y test. Es la forma correcta de medir si el modelo funciona con personas nuevas.

3. Aumento de datos

Rotaciones pequeñas, espejos, cambios de brillo y recortes. Ayuda a que el modelo no dependa de una cámara o iluminación concreta.

4. Pesos de clase o umbral ajustado

Compensar el desbalance y elegir un umbral que equilibre mejor falsas alarmas y falsos negativos según el costo real.

5. Detección y recorte de cara

Usar Haar Cascade, MediaPipe o MTCNN para recortar el rostro antes de clasificar. Así los ojos ocupan más píxeles y el modelo ve mejor.

6. Métricas de ojos y boca

Calcular EAR (Eye Aspect Ratio) y MAR (Mouth Aspect Ratio) con puntos faciales. Son señales geométricas muy útiles para somnolencia.

7. Modelo temporal

Usar una CNN + LSTM, un Transformer de video o un modelo 3D para analizar secuencias, no solo frames aislados.

8. Transfer learning

Partir de MobileNetV3, EfficientNetB0 o ResNet y ajustar las últimas capas. Suele dar mejor precisión con menos datos.

9. Calibración

Ajustar las probabilidades para que un 0.8 signifique realmente 80% de probabilidad. Ayuda a elegir umbrales con criterio.

10. Explicabilidad

Usar Grad-CAM para ver qué zonas de la imagen influyen en la decisión. Es útil para detectar si el modelo mira la cara o el fondo.

La mejora más importante para un uso real

No es solo “más precisión”: es **validar con personas y condiciones nuevas**. Un modelo puede tener 95% en un dataset y fallar en un auto real si nunca vio esa cámara, esa luz o esa persona. La validación externa es lo que separa una demo de un sistema confiable.

13. Glosario para principiantes

Términos que aparecen en el proyecto, explicados sin tecnicismos innecesarios.

Término	Significado sencillo
Inteligencia artificial	Programas que realizan tareas que normalmente asociamos con la inteligencia humana, como reconocer imágenes.
Machine learning	Un programa que aprende patrones a partir de ejemplos, en lugar de seguir solo reglas escritas a mano.
Deep learning	Machine learning con redes neuronales de muchas capas.
CNN	Red neuronal convolucional: una red especializada en imágenes.
Píxel	El cuadradito más pequeño de una imagen.
Canal	Una “capa de color” de la imagen: rojo, verde o azul.
Tensor	Una caja de números con varias dimensiones. Una imagen es un tensor de 3 dimensiones.
Kernel / filtro	Una matriz pequeña de números que se desliza por la imagen para detectar un patrón.
Convolución	La operación de aplicar un filtro sobre la imagen.
Stride	Cuántos píxeles se mueve el filtro en cada paso.
Padding	Relleno en los bordes para controlar el tamaño de la salida.
ReLU	Función que deja los positivos y convierte los negativos en cero.
BatchNorm	Normalización de las activaciones para estabilizar el entrenamiento.
Pooling	Resumen de una región de la imagen para reducir su tamaño.
MaxPooling	Quedarse con el valor máximo de cada región.
Global Average Pooling	Promediar cada mapa de características para obtener un solo número.
Dropout	Apagar neuronas al azar durante el entrenamiento para evitar memorización.
Dense	Capa totalmente conectada; combina características para decidir.
Sigmoid	Función que convierte cualquier número en un valor entre 0 y 1.
Probabilidad	Qué tan probable es que algo ocurra, de 0 a 1.
Umbral	El valor a partir del cual tomamos una decisión. Aquí, 0.50.
Época	Una pasada completa por todos los datos de entrenamiento.
Batch	Un grupo pequeño de ejemplos procesados juntos.
Pérdida (loss)	Un número que mide el error del modelo.
Retropropagación	El proceso de calcular cómo ajustar los pesos para reducir el error.
Adam	Un optimizador que ajusta los pesos de forma adaptativa.
Sobreajuste	Cuando el modelo memoriza el entrenamiento y falla con datos nuevos.
Accuracy	Porcentaje total de aciertos.

Precision	De lo que el modelo marcó como positivo, qué porcentaje era correcto.
Recall	De todos los positivos reales, qué porcentaje detectó.
F1-score	Equilibrio entre precision y recall.
ROC	Gráfica que muestra el trade-off entre detectar positivos y generar falsos positivos.
AUC	Área bajo la curva ROC; mide la capacidad de separar clases.
Matriz de confusión	Tabla que cruza las clases reales con las predicciones.
TP / TN / FP / FN	Verdadero positivo, verdadero negativo, falso positivo y falso negativo.
Buffer temporal	Contador de frames consecutivos con somnolencia antes de alarmar.
Inferencia	Usar un modelo ya entrenado para hacer una predicción.

14. Resumen para recordar

Si mañana solo recuerdas una página de este documento, que sea esta.

Las 10 ideas clave

1. **El modelo recibe una imagen de 64 × 64 y devuelve una probabilidad entre 0 y 1.** Si es ≥ 0.5, la app alerta somnolencia.
2. **Es una CNN pequeña:** 11 capas y 36,737 parámetros. Está diseñada para ser rápida y liviana.
3. **Los filtros son matrices de 3 × 3 que se deslizan por la imagen.** La red aprende sus números; nadie los escribe a mano.
4. **Los primeros filtros detectan bordes y contrastes; los siguientes, formas; los últimos, patrones de cansancio.**
5. **Se entrenó con el dataset DDD, pero con un subconjunto de hasta 9,600 imágenes.** No se usó el dataset completo.
6. **El entrenamiento duró 168.48 segundos y completó 6 épocas.** La exactitud de validación terminó en 86.11%.
7. **En el test de 1,152 imágenes: accuracy 86.46%, precision 81.35%, recall 96.56%, F1 88.31%, AUC 94.86%.**
8. **Detecta muy bien la somnolencia (solo 21 falsos negativos), pero genera muchas falsas alarmas (135).** El buffer de 10 frames ayuda a reducirlas.
9. **El modelo no recorta la cara ni analiza video de forma nativa.** Eso limita su uso en condiciones reales.
10. **No es un sistema certificado de seguridad.** Es una demo académica para entender Deep Learning y visión por computadora.

La explicación en 30 segundos

> *“Una CNN aprende mirando muchas fotos de personas despiertas y somnolientas. Con filtros pequeños va detectando bordes, luego formas y luego patrones de cansancio. Al final combina todo y dice una probabilidad. En este proyecto acierta el 86.46% de las veces, detecta el 96.56% de los somnolientos, pero da falsas alarmas en el 24.91% de los despiertos. Por eso es una demo, no un sistema de seguridad certificado.”*

Archivos del proyecto

Archivo	Qué contiene
`train.py`	Pipeline de carga, arquitectura CNN, entrenamiento, evaluación y exportación de métricas.
`app.py`	Aplicación Streamlit: carga de imágenes, cámara, métricas y explicación interactiva.
`model_ddd.keras`	Modelo entrenado, listo para cargar y predecir.
`metrics.json`	Métricas, matriz de confusión, historia de entrenamiento y resumen de arquitectura.
`training_curves.png`	Curvas de accuracy y loss por época.
`confusion_matrix.png`	Matriz de confusión normalizada.
`roc_curve.png`	Curva ROC y AUC.

Cómo reproducir

```
pip install -r requirements.txt
python train.py --epochs 6 --batch_size 64 --max_samples 9600
streamlit run app.py
```

Documento educativo generado para el Proyecto Sprint 2 · GuardDrive AI · 23 de octubre de 2026.
Basado en `train.py`, `app.py`, `metrics.json` y las gráficas del repositorio.